

Edeviser — Complete Architecture, Student Profile & System Audit

Prepared: July 12, 2026

Scope: Full-stack codebase (React 18 + TypeScript + Vite 6 + Supabase), live Supabase project `cdlgtbvxlxjpcddjazzx` (ap-northeast-1, Postgres 17.6.1), Vercel deployment, all 6 roles (admin, coordinator, teacher, student, parent, public/auth), OBE + Gamification + AI subsystems.

Method: Direct codebase inspection (routers, hooks, components, edge functions, migrations), live Supabase MCP inspection (tables, RLS policies, functions/triggers, extensions, edge function deployment status, security/performance advisors, live edge-function logs), and reconciliation against **nine prior internal audit documents** dated March–August 2026. Every finding below is labeled **[VERIFIED-LIVE]** (confirmed against the running system or current source on this date), **[VERIFIED-CODE]** (confirmed by reading current source, not yet exercised live), or **[RECOMMENDATION]** (an inference/suggestion, not an observed fact). Where a prior audit's finding has been superseded, it is explicitly marked **RESOLVED** or **STALE**.

Table of Contents

- Executive Summary
- System Architecture Overview
- Frontend Audit
- Backend Audit
- Supabase Database Audit
- ER Diagrams & Relationship Maps
- API & Data Flow Diagrams
- Student Profile Deep Analysis
- UX & Product Evaluation
- Simplification Recommendations
- AI Integration Review
- Performance & Scalability Analysis
- Security & RLS Review
- Code Quality & Technical Debt Assessment
- Prioritized Improvement Roadmap
- Appendices — Historical Audit Reconciliation

1. Executive Summary

Edeviser is a mature, richly-seeded OBE (Outcome-Based Education) + Gamification platform for higher education, built on React 18/TypeScript/Vite 6 and Supabase (Postgres, Auth, Realtime, Storage, 61 Edge Functions). The live database holds **131 tables** (100% RLS-enabled), real production-shaped data (124+ profiles, 859+ grades, 2,577+ evidence records, 4,150+ XP transactions, 62 badges awarded), and a working core academic loop: grade submission → evidence → CLO/PLO/ILO attainment rollup → student-facing progress, all functioning correctly on live data.

This audit found the platform in **substantially better health than several prior internal audits suggest** — a progression of real fixes between April and August 2026 resolved the badge subsystem, the habit-table split, the Sentry consent-gate bypass, several cross-tenant RLS gaps, and the previously-broken build. Those historical findings are documented in full in **Appendix 16** and are marked resolved rather than silently dropped.

However, this audit also found **one severe, currently-live production defect** that no prior document had confirmed via live logs: **most cron-driven Edge Functions are 401-ing at the Supabase gateway right now** (Finding B-1, Section 4.3), silently disabling streak resets, at-risk predictions, notification digests, the perfect-day nudge, and the exam-period reminder. This is the single highest-priority fix in this report.

The **Student Profile** — this audit's primary focus — is architecturally sound at its center (a single consolidated dashboard RPC, correct attainment-scope consistency across every student-facing read) but has accumulated **five fully-built, tested, and currently unreachable features** (a tiered badge page, a CLO drill-down view, a Bloom's-ordered learning path, a goal-suggestion panel, and a legacy journal editor with CLO-linking) and **meaningful data duplication** (XP total, CLO mastery, and badges are each independently queried in 3–5 places rather than sharing one source of truth). Section 8–10 give the full breakdown and a concrete, non-destructive simplification plan.

Top findings at a glance

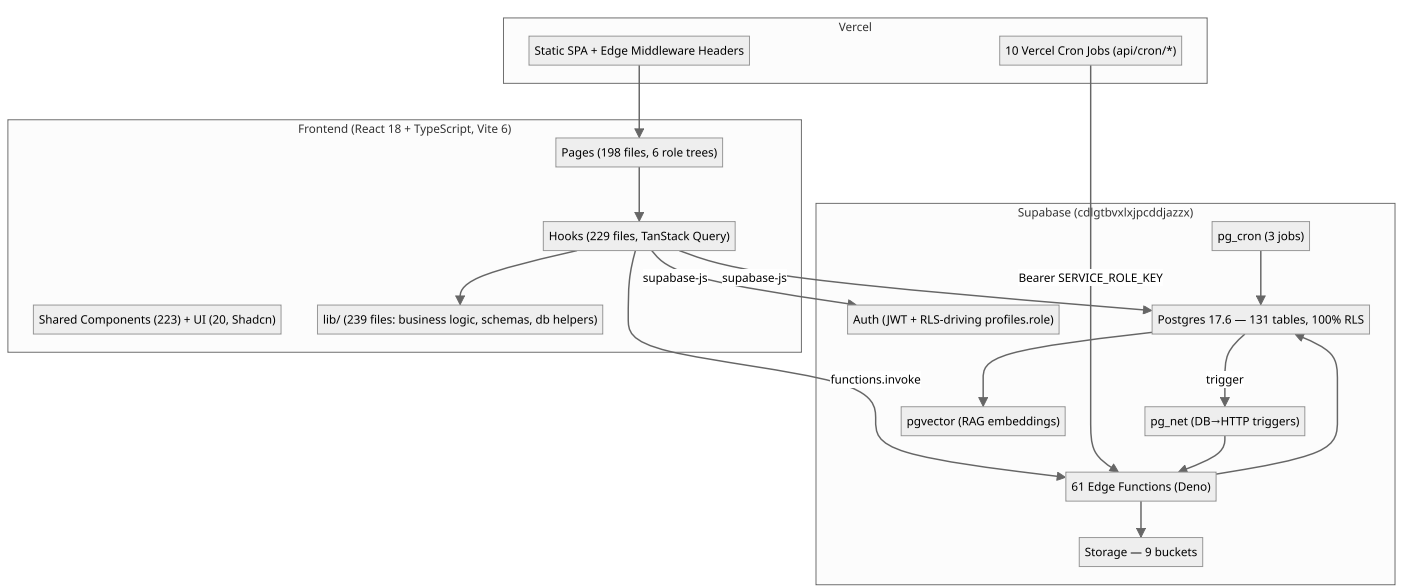
#	Finding	Severity	Status
B-1	Cron→Edge Function calls 401 at the gateway (non-JWT service-role key vs <code>verify_jwt=true</code>) — streak reset, at-risk crons, notification digest, perfect-day nudge, exam reminder all silently dead	● Critical	Live now
H-1	<code>capture_active_semester_snapshots</code> SECURITY DEFINER function has no role/institution guard — callable by any authenticated (and anon) user, writes cross-institution data	● High (security)	Live now
H-2	Admin PLO attainment heatmap card renders a static empty <code><p></code> — no data hook, no drill-down (component + hook exist but are disconnected)	● High	Live now (spec'd fix pending)
H-3	<code>outcome_mappings</code> weight-sum-to-100 invariant not enforced anywhere (DB or Zod)	● High	Live now
H-4	Widespread <code>multiple_permissive_policies</code> RLS overhead across dozens of tables	● Medium (perf)	Live now, gated remediation plan exists

#	Finding	Severity	Status
—	Badge subsystem dead end-to-end (0 badges awarded)	Was Critical	RESOLVED — 62 badges live, 4,150+ XP transactions, 71 students with XP > 0
—	Only 1–2 of ~54 Edge Functions deployed to production	Was Critical	RESOLVED — 61 functions confirmed ACTIVE live
—	Zero Storage buckets configured	Was Critical	RESOLVED — 9 buckets live, correctly scoped (only avatars public)
—	Sentry initializes before cookie-consent gate	Was High	RESOLVED — verified in current App.tsx / main.tsx / sentry.ts
—	database.ts truncated, build failing	Was Critical	RESOLVED — current file is a complete, valid 7,590-line generated file

Full detail, evidence, and the complete historical reconciliation are in the sections that follow.

2. System Architecture Overview

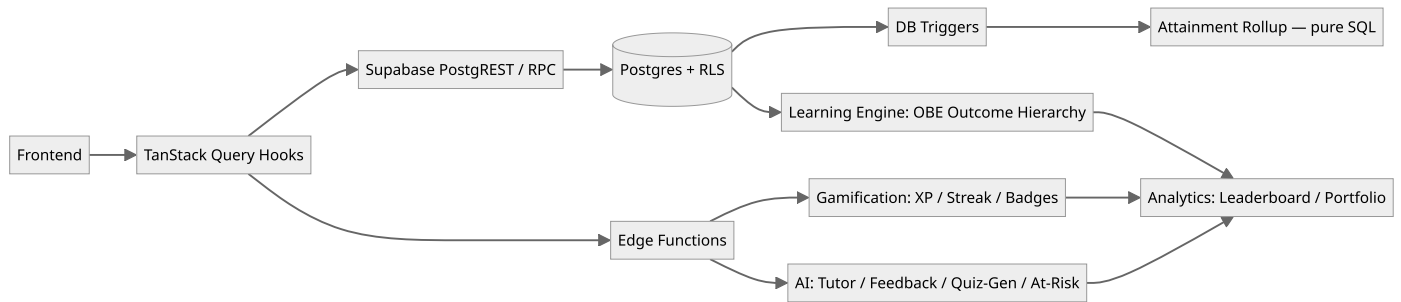
2.1 High-level architecture



2.2 Tech stack (confirmed from package.json + source)

React 18, TypeScript (strict), Vite 6, Tailwind CSS v4 + Shadcn/ui, React Router v7, TanStack Query v5 + TanStack Table, React Hook Form + Zod, Framer Motion, Recharts, dnd-kit, date-fns, Sonner, nuqs, Zustand (single store), i18next (en/ar, RTL), @sentry/react, canvas-confetti. Backend: Supabase (Postgres, Auth, Realtime, Storage, Edge Functions/Deno), Vercel (hosting + cron), pg_cron/pg_net/pgvector extensions, Resend (transactional email, Edge-Function-only).

2.3 Domain layers (as implemented)



This matches the reference C4 diagrams in docs/architecture/SYSTEM-ARCHITECTURE.md at the container level; the notable **drift** from that reference document is documented per-finding in Sections 5, 7, and 13 (e.g., the attainment rollup is implemented as a pure-SQL trigger, not the calculate-attainment-rollup Edge Function the design doc describes as primary — the Edge Function still exists and is deployed but is superseded by the DB trigger for the live write path).

3. Frontend Audit

3.1 Routing structure [VERIFIED-CODE]

`src/router/AppRouter.tsx` (864 lines) uses a **single-wrap-per-role** `RouteGuard` pattern — each role's entire route subtree is wrapped once (`<Route path="/{role}/*" element={<RouteGuard allowedRoles={["{role}"]}><{Role}Layout/></RouteGuard></>`), not per-page. This is a clean, low-risk pattern: there is exactly one gate per role to reason about, and it is backed by RLS as the real security boundary (Section 13). One deliberate outlier: `/student/focus/:sessionId` is guarded individually outside `StudentLayout` since Focus Mode is a full-screen, sidebar-free experience.

Route group	Approx. routes	Notes
Public/auth	8	login, signup, accept-invite, reset/update password, public portfolio, terms, privacy
Admin	~38	users, programs, outcomes, courses, semesters, departments, fees, imports, surveys, badges/spotlight, marketplace, audit log
Coordinator	~14	PLOs, curriculum matrix, CQI, course-file, gap-analysis, trends, cohort-comparison
Teacher	~40	CLOs, rubrics, assignments, grading, baseline tests, quiz generation/analytics, teams, challenges
Student	~45	dashboard, progress, journal, habits, planner, marketplace, challenges, tutor, portfolio
Parent	8	dashboard, children, progress, attendance, planner, profile
Shared (cross-role)	—	<code>CalendarView</code> , <code>TimetableView</code> , <code>ProfilePage</code> , <code>OutcomeChainView</code> , <code>NotificationPreferences</code> , <code>SessionManagement</code> — reused, not duplicated per role

Total distinct lazy-loaded route components: ~150+.

3.2 Pages inventory [VERIFIED-CODE]

198 `.tsx` files under `src/pages/`. A consistent **"New UI" migration pattern** is in progress (tracked in `.kiro/specs/ui-prototype-migration/`): several stable filenames are thin re-export shims pointing at a `*New` implementation —

`AdminDashboard.tsx` → `AdminDashboardNew` , `CurriculumMatrixPage.tsx` → `CurriculumMatrixPageNew` , `StudentCoursesPage.tsx` → `StudentCoursesNew` , `StudentJournalPage.tsx` → `StudentJournalNew` , `StudentPortfolio.tsx` → `StudentPortfolioNew` , `StudentProgressPage.tsx` → `StudentProgressNew` , `XPHistory.tsx` → `XPHistoryNew` .

This is a reasonable, low-risk migration technique (stable public route → swappable implementation), but it leaves the **old implementations orphaned on disk** rather than deleted. Confirmed via exhaustive grep across `src/` — **the following files have zero references anywhere**** except their own unit tests:

Orphaned file	Why it's dead
<code>admin/settings/AdminProfilePage.tsx</code>	superseded by shared <code>ProfilePage</code>
<code>coordinator/settings/CoordinatorProfilePage.tsx</code>	superseded by shared <code>ProfilePage</code>
<code>teacher/settings/TeacherProfilePage.tsx</code>	superseded by shared <code>ProfilePage</code>
<code>admin/settings/WellnessXpSettingsPage.tsx</code>	no route, no import anywhere
<code>student/quizzes/QuizAttemptPage.tsx</code>	student quiz flow uses <code>AdaptiveQuizSession</code> / <code>PostQuizReview</code> instead
<code>teacher/quizzes/QuizBuilder.tsx</code>	956 lines — router uses <code>QuizForm</code> instead
<code>teacher/gradebook/GradeCategoryManager.tsx</code>	not imported by <code>GradebookView</code> or router
<code>student/leaderboard/TeamLeaderboard.tsx</code>	distinct from the used <code>TeamLeaderboardView.tsx</code>
<code>teacher/dashboard/GradingStats.tsx</code>	distinct from the used <code>useGradingStats.ts</code> hook
<code>student/progress/CLOProgress.tsx</code>	fully built + tested, not routed (see Section 8)
<code>student/progress/LearningPath.tsx</code>	fully built + tested, not routed (see Section 8)

3.3 Hooks inventory [VERIFIED-CODE]

229 flat files in `src/hooks/` , **no subfolders** despite spanning every domain (admin/coordinator/teacher/student/parent/gamification/wellness/tutor). This is a growth of the same flat-directory issue an earlier architecture audit flagged at 82 files — the directory nearly tripled without being restructured. Notable clusters:

- **Dashboard aggregate pairs** across all 5 roles (e.g. `useStudentDashboard` + `useStudentDashboardAggregate`) — the *Aggregate* variants are the newer, consolidated-fetch versions (confirmed for student; see Section 8). The non- *Aggregate* siblings should be audited for deadness in the same pass that resolves the page-level `New` shims.
- **Six distinct leaderboard hooks** (`useLeaderboard` , `useChallengeLeaderboard` , `useLeagueLeaderboard` , `useMostImprovedLeaderboard` , `usePersonalBestLeaderboard` , `useTeamLeaderboard`) — likely legitimately distinct views, but a candidate for a shared base hook if their query shapes overlap.
- A single `__tests__` subfolder is the only structure present.

3.4 Shared components [VERIFIED-CODE]

- `src/components/ui/` — 20 files, a lean Shadcn primitive set, no bloat.

- `src/components/shared/` — 223 flat files. The `*New` dashboard components (`AdminDashboardNew` , `CoordinatorDashboardNew` , `StudentDashboardNew` , `TeacherDashboardNew` , `ParentDashboardNew` , `CoordinatorOutcomeAttainmentNew` , `CoordinatorProfileNew`) are the **live, correctly-wired implementations** behind the page shims (Section 3.2) — not orphans, but the `New` suffix should be dropped once the migration spec fully lands.
- Confirmed orphans:** `LanguageSelector.tsx` (duplicate of the actually-used `LanguageSwitcher.tsx`); `PLODrillDownDialog.tsx` and its hook `useAdminPLOHeatmap.ts` (zero references anywhere) — this directly corroborates Finding **H-2** (Section 4.4): the Admin PLO heatmap component/hook pair exists in the codebase but was disconnected from the `AdminDashboard` , which now renders a static empty `<p>` in its place. A fix is already specified in `.kiro/specs/qa-partner-review-remediation/requirements.md` (Requirement 7, priority P3) but not yet implemented.

3.5 Business logic (`src/lib/`) [VERIFIED-CODE]

239 files across the root plus `db/` (5 files — a dedicated Supabase column/type-safety layer: `castGuard.ts` , `insertColumns.ts` , `pickColumns.ts` , `resolveName.ts` , `schemaContract.ts`), `schemas/` (75 files, one Zod schema per entity, matching the project convention of centralizing validation), and `tours/` (5 files). Domain logic is correctly kept out of components/hooks (`xpClient.ts` , `xpLevelCalculator.ts` , `auditLoggers.ts` , `perfectDay.ts` , `streakCalculator.ts` , `attainmentClassifier.ts` , `badgeDefinitions.ts` , `bloomsClimb.ts` , `leagueTiers.ts` , `masteryRecovery.ts` , etc.), matching the architecture guardrails. The root of `lib/` is flat with 150+ loose files mixing every domain — a candidate for domain subfolders (`lib/gamification/` , `lib/wellness/` , `lib/tutor/`) as the codebase continues to grow, though no actually-duplicated logic was confirmed in this pass (flagged for a follow-up: possible overlap between `leagueTiers` / `leagueTierCalculator.ts` and `mostImprovedLeaderboard.ts` / `personalBestLeaderboard.ts`).

3.6 Providers, router, stores, types [VERIFIED-CODE]

- `src/providers/` — 5 files, each single-concern (`AuthProvider` , `FocusModeProvider` , `ImpersonationProvider` , `LanguageProvider` , `ThemeProvider`).
- `src/stores/` — only **one** Zustand store (`tourStore.ts` , guided-tour state) despite Zustand being a listed dependency; the rest of client state lives in TanStack Query cache or local component state. Not a defect, just worth noting for anyone expecting a larger global-state surface.
- `src/types/` — `app.ts` (653 lines, a monolithic domain-type barrel), `database.ts` (7,590 lines, auto-generated — **do not hand-edit**, see the workspace's `types-regeneration` rule), `habits.ts` , `pagination.ts` , `planners.ts` .

3.7 Code-quality signals — large files [VERIFIED-CODE]

File	Lines	Assessment
<code>src/types/database.ts</code>	7,590	Expected — auto-generated, valid, current (see Section 16.1: this was previously reported truncated/broken and is now confirmed intact).
<code>src/router/AppRouter.tsx</code>	864	Monolithic — a reasonable refactor target: split into per-role route files (<code>adminRoutes.tsx</code> , <code>studentRoutes.tsx</code> , ...) composed in <code>AppRouter.tsx</code> . Not urgent; the file is readable and the guard pattern is sound.
<code>src/types/app.ts</code>	653	Single barrel for all domain types — a split-by-domain candidate if it keeps growing, not currently a problem.
<code>src/lib/queryKeys.ts</code>	459	Appropriate size for a single source-of-truth key registry covering ~150 hooks — not a code smell.
<code>src/pages/teacher/quizzes/QuizBuilder.tsx</code>	956	Dead file (Section 3.2) — a deletion candidate, not a refactor candidate.

Recommendation: a single follow-up PR should delete the confirmed-orphaned files listed in 3.2/3.4 (`AdminProfilePage` , `CoordinatorProfilePage` , `TeacherProfilePage` , `WellnessXpSettingsPage` , `QuizAttemptPage` , `QuizBuilder` , `GradeCategoryManager` , `TeamLeaderboard.tsx` , `GradingStats.tsx` , `CLOProgress.tsx` , `LearningPath.tsx` , `LanguageSelector.tsx` , `PLODrillDownDialog.tsx` + `useAdminPLOHeatmap.ts`) and run the full pre-push gate (`npm run lint && npx tsc --noEmit && npm test`) to confirm zero call sites break — **unless** `CLOProgress.tsx` / `LearningPath.tsx` are instead wired back in per Section 10's recommendation, which is the preferred outcome since they represent real, tested functionality rather than pure dead weight.

4. Backend Audit

4.1 Vercel serverless functions (`api/`) [VERIFIED-CODE]

`api/` contains only `cron/` (10 files) and `_utils/auth.ts` — there is no general-purpose REST API layer; all data access goes through Supabase PostgREST/RPC directly from the frontend, or through Edge Functions for privileged/server-side operations. This is architecturally clean and matches the "components never call Supabase directly, hooks do" convention, extended correctly to "no bespoke backend API needed."

`api/_utils/auth.ts` provides two helpers used by every cron file:

- `verifyCronSecret(req, res)` — validates `Authorization: Bearer ${CRON_SECRET}` on inbound requests from Vercel's scheduler.
- `invokeEdgeFunction(functionName, body)` — calls `${SUPABASE_URL}/functions/v1/${functionName}` with `Authorization: Bearer ${SUPABASE_SERVICE_ROLE_KEY}` .

4.2 Vercel Cron jobs [VERIFIED-LIVE]

`vercel.json` schedules 10 crons:

Path	Schedule	Target Edge Function
<code>/api/cron/streak-risk</code>	<code>0 20 *</code>	<code>streak-risk-cron</code>
<code>/api/cron/weekly-summary</code>	<code>0 8 1</code>	<code>weekly-summary-cron</code>
<code>/api/cron/compute-at-risk</code>	<code>0 2 *</code>	<code>compute-at-risk-signals</code>

Path	Schedule	Target Edge Function
/api/cron/perfect-day-prompt	0 18 *	perfect-day-prompt
/api/cron/streak-reset	0 0 *	process-streak (midnight_reset)
/api/cron/leaderboard-refresh	0 12 *	leaderboard-refresh
/api/cron/ai-at-risk-prediction	0 3 *	ai-at-risk-prediction
/api/cron/notification-digest	0 20 *	notification-digest
/api/cron/exam-period-notify	0 8 *	exam-period-notify
/api/cron/warm-ping	0 6 *	health

The **8am/9am exam-period-notify mismatch** reported by a prior audit is confirmed **RESOLVED** — the schedule now correctly reads `0 8 *` matching the function's own documented expectation. The **warm-ping cron is a new addition** since the last performance audit and is a real, working implementation of that audit's Tier-1 recommendation: it gates on an active-hours window (06:00-23:00 UTC, env-configurable) and documents in its own comments the Vercel Hobby-plan constraint that sub-daily cron schedules are rejected at deploy time — a well-reasoned, low-risk implementation.

4.3 Finding B-1 (Critical): cron→Edge Function calls are 401-ing at the gateway right now [VERIFIED-LIVE]

A prior product audit (June 2026, §11) identified a systemic root cause: this Supabase project's keys are the modern non-JWT `sb_...` format (confirmed via `get_publishable_keys : sb_publishable_-xh23FT...` alongside a legacy JWT-format anon key). Supabase Edge Functions deployed with the default `verify_jwt=true` have the **platform gateway** validate the `Authorization` header as a JWT before the function handler ever runs. A non-JWT `sb_secret_...` service-role key fails that gateway check and produces an **HTTP 401 before the function's own in-handler authorization logic executes** — regardless of how correct that in-handler logic is.

An August 2026 remediation report claimed this was fixed for `award-xp` and `check-badges` (redeployed with `verify_jwt=false` + an internal `x-internal-auth` header check instead). **This audit independently confirmed via live Edge Function logs (`get_logs` , `service=edge-function`) that the fix was scoped correctly to those two functions but was never extended to the cron-invoked functions**, which still 401 today:

```
POST | 401 | .../functions/v1/perfect-day-prompt
POST | 401 | .../functions/v1/leaderboard-refresh
POST | 401 | .../functions/v1/exam-period-notify
POST | 401 | .../functions/v1/ai-at-risk-prediction
POST | 401 | .../functions/v1/compute-at-risk-signals
POST | 401 | .../functions/v1/process-streak (the midnight_reset invocation)
POST | 401 | .../functions/v1/notification-digest
POST | 401 | .../functions/v1/streak-risk-cron
```

For contrast, in the **same log window**, `award-xp`, `check-badges`, `check-login-rate`, and `health` all returned `200` — these are exactly the functions that were redeployed with `verify_jwt=false`.

Live impact today: the nightly streak reset, the 2am/3am at-risk-signal computation and AI at-risk prediction, the notification digest, the 6pm perfect-day nudge, the leaderboard refresh, the exam-period reminder, and the evening streak-risk warning are **all silently non-functional in production**. None of these failures surface to end users or admins as an error — they simply never run. This is the single highest-priority fix in this report.

Fix (proven pattern already in the codebase): redeploy each affected function with `verify_jwt=false` and add the same in-handler `x-internal-auth` /service-role check already used by `award-xp` and `check-badges`. This is a deployment-configuration change plus a small, mechanical code change per function — not a redesign.

4.4 Finding H-1 (High, security): capture_active_semester_snapshots unguarded [VERIFIED-LIVE]

This `SECURITY DEFINER` function (confirmed by direct source read) has **no role check and no institution filter at all** — it aggregates attainment data across **all institutions unconditionally** and writes into `outcome_attainment_snapshots`. Supabase's security advisor confirms it is flagged `anon_security_definer_function_executable` and `authenticated_security_definer_function_executable`, meaning it is callable by any signed-in user (and possibly anonymous callers) via `/rest/v1/rpc/capture_active_semester_snapshots`. This is a live, exploitable cross-tenant data-integrity issue: any authenticated user at Institution A can trigger a write that aggregates and stores Institution B's data.

For contrast, two functions a prior security audit flagged as similarly unguarded — `delete_department_if_no_programs` and `get_earn_spend_ratio` — were independently re-read this audit and are now **properly guarded** (role/institution checks present). Those two are marked **RESOLVED** in Appendix 16.

Fix: apply the same pattern already used correctly elsewhere in the codebase (`get_wellness_aggregate_stats`, `get_leaderboard_page`, etc. — see the workspace's own `supabase-patterns` steering rule, which documents this exact remediation): either `REVOKE EXECUTE ... FROM PUBLIC; GRANT ... TO authenticated;` with an explicit role check, or add an internal fail-closed guard comparing the caller's `auth_institution_id()` against the target institution and raising an exception on mismatch.

4.5 Other SECURITY DEFINER functions flagged by advisors — not findings [VERIFIED-LIVE]

The remaining functions flagged `anon_security_definer_function_executable` / `authenticated_security_definer_function_executable` were individually read and are safe by design: `auth_institution_id`, `auth_user_role`, `auth_user_status` (intentional auth-helper predicates), and `get_coordinator_accreditation_readiness`, `get_wellness_aggregate_stats`, `get_leaderboard_page`, `fan_out_announcement_notifications`, `send_teacher_nudge`, `get_badge_spotlight`, `get_historical_evidence`, `get_student_dashboard`, `get_teacher_dashboard`, `parent_has_verified_link`, `is_portfolio_publicly_accessible`, `is_student_in_my_institution`, `portfolio_public_access`, `student_enrolled_in_team_course`, `team_i_captain`, `team_in_course_i_teach`, `team_in_my_institution`, `consume_invitation`, `get_invitation_by_token`, `course_material_institution` — each already contains an internal role/institution/ownership check.

4.6 Edge Function inventory [VERIFIED-LIVE]

61 Edge Functions are deployed and ACTIVE in the live project (confirmed via `list_edge_functions`, versions ranging 5–15, i.e., iteratively redeployed, not first-time deploys). This directly **resolves** the "only 1–2 of ~43–54 functions deployed" finding present in three separate prior audit documents (April 1, April 16, April 30 dates) — those findings are now stale (Appendix 16).

Notable categories from source inspection + the June product audit's classification (independently spot-checked, not fully re-derived):

- Core wired:** `award-xp`, `check-badges`, `process-streak`, `check-login-rate`, `chat-with-tutor`, `generate-quiz-questions`, `bulk-import-users`, `process-onboarding`, `health`.
- Cron-driven, currently broken (Section 4.3):** `perfect-day-prompt`, `leaderboard-refresh`, `exam-period-notify`, `ai-at-risk-prediction`, `compute-at-risk-signals`, `notification-digest`, `streak-risk-cron`, `weekly-summary-cron` (not directly observed 401 in the sampled log window but shares the same invocation pattern — should be verified in the same remediation pass).
- AI-powered:** `ai-feedback-draft`, `ai-module-suggestion`, `embed-course-material` (RAG indexing — provider-agnostic since the August remediation, but pending an `EMBEDDINGS_API_KEY` secret; **not independently verifiable** by this audit since no MCP tool exposes Edge Function secrets — reported per the source document's own admission as an ops-pending step, not a code defect).
- Orphaned (no caller found in this or prior audits):** `ai-module-suggestion`'s consumer (`GoalSuggestionPanel`) is itself orphaned on the frontend (Section 8) even though the function is deployed and reachable — a frontend-side gap, not a backend defect.

4.7 Storage buckets [VERIFIED-LIVE]

9 buckets exist live, correctly scoped: `avatars` (public — profile photos, appropriate), and `submissions`, `course-materials`, `accreditation-reports`, `tutor-attachments`, `reports`, `transcripts`, `announcement-attachments`, `session-evidence` (all private). This **resolves** the "zero storage buckets" finding present in two prior audit documents (Appendix 16).

5. Supabase Database Audit

5.1 Tables [VERIFIED-LIVE]

131 tables, 100% RLS-enabled, every table policy-backed (0 tables enabled-without-policy — a genuine positive finding, confirmed live). This is a large growth from the 47/57/67-table counts in earlier audit documents (Appendix 16) as the platform added onboarding, wellness, mastery-recovery, question-bank, teams, marketplace, tutor, and planner subsystems.

Representative table groups (not exhaustive — see `docs/architecture/SUPABASE-COLUMNS-DATATYPES.md` for the full column-level reference, which remains broadly accurate for schema shape though table *count* has grown since it was generated):

Domain	Representative tables
Multi-tenancy / org	<code>institutions</code> , <code>departments</code> , <code>programs</code> , <code>semesters</code> , <code>institution_settings</code>
People	<code>profiles</code> , <code>parent_student_links</code>
Curriculum (OBE)	<code>learning_outcomes</code> (ILO/PLO/CLO), <code>outcome_mappings</code> , <code>outcome_attainment</code> , <code>outcome_attainment_snapshots</code> , <code>rubrics</code> , <code>rubric_criteria</code>
Teaching	<code>courses</code> , <code>course_sections</code> , <code>course_modules</code> , <code>course_materials</code> , <code>assignments</code> , <code>submissions</code> , <code>grades</code> , <code>evidence</code>
Assessment	<code>quizzes</code> , <code>quiz_questions</code> , <code>quiz_attempts</code> , <code>question_bank</code> , <code>question_analytics</code> , <code>baseline_attainment</code> , <code>baseline_test_config</code>
Gamification	<code>student_gamification</code> , <code>badges</code> , <code>xp_transactions</code> , <code>xp_events</code> , <code>habit_logs</code> , <code>wellness_habit_logs</code> , <code>teams</code> , <code>team_gamification</code> , <code>social_challenges</code> , <code>challenge_participants</code>
Student-support	<code>journal_entries</code> , <code>student_profiles</code> , <code>onboarding_*</code> , <code>micro_assessment_schedule</code> , <code>starter_week_sessions</code> , <code>goal_suggestions</code> , <code>mastery_recovery_pathways</code>
AI/Tutor	<code>tutor_conversations</code> , <code>tutor_messages</code> , <code>course_material_embeddings</code> (pgvector)
Accreditation	<code>surveys</code> , <code>survey_questions</code> , <code>survey_responses</code> , <code>cqi_action_plans</code> , <code>program_accreditations</code>
Ops	<code>audit_logs</code> , <code>notifications</code> , <code>student_activity_log</code> , <code>login_attempts</code>

5.2 Extensions [VERIFIED-LIVE]

`plpgsql`, `pg_stat_statements`, `uuid-oss`, `pgcrypto`, `pg_net` (public schema — see below), `pg_cron`, `pg_graphql`, `supabase_vault`, plus `citext` and `vector` (both flagged by the advisor as living in `public` rather than a dedicated schema — low-severity, standard hardening recommendation, not urgent).

5.3 Functions & triggers [VERIFIED-LIVE]

The core attainment cascade is implemented as a **pure-SQL DB trigger** (`trigger_attainment_rollup`, rewritten in migration `20260520070203_rewrite_attainment_trigger_pure_sql.sql`), not the `calculate-attainment-rollup` Edge Function that earlier design docs describe as primary. The Edge Function still exists, is deployed, and is invoked by nothing in the live write path for grading — this is an intentional supersession per the August remediation report's own note ("stays disconnected per 3.7"), not a bug, but the reference architecture document should be updated to reflect this as the current source of truth.

5.4 Row Level Security [VERIFIED-LIVE, partially]

131/131 tables have RLS enabled with at least one policy — confirmed via advisors and consistent across every document from April onward. Standard patterns match the workspace's own `supabase-patterns` steering rule (admin full-institution CRUD, student-own-row read, parent-verified-link read, append-only insert-only for

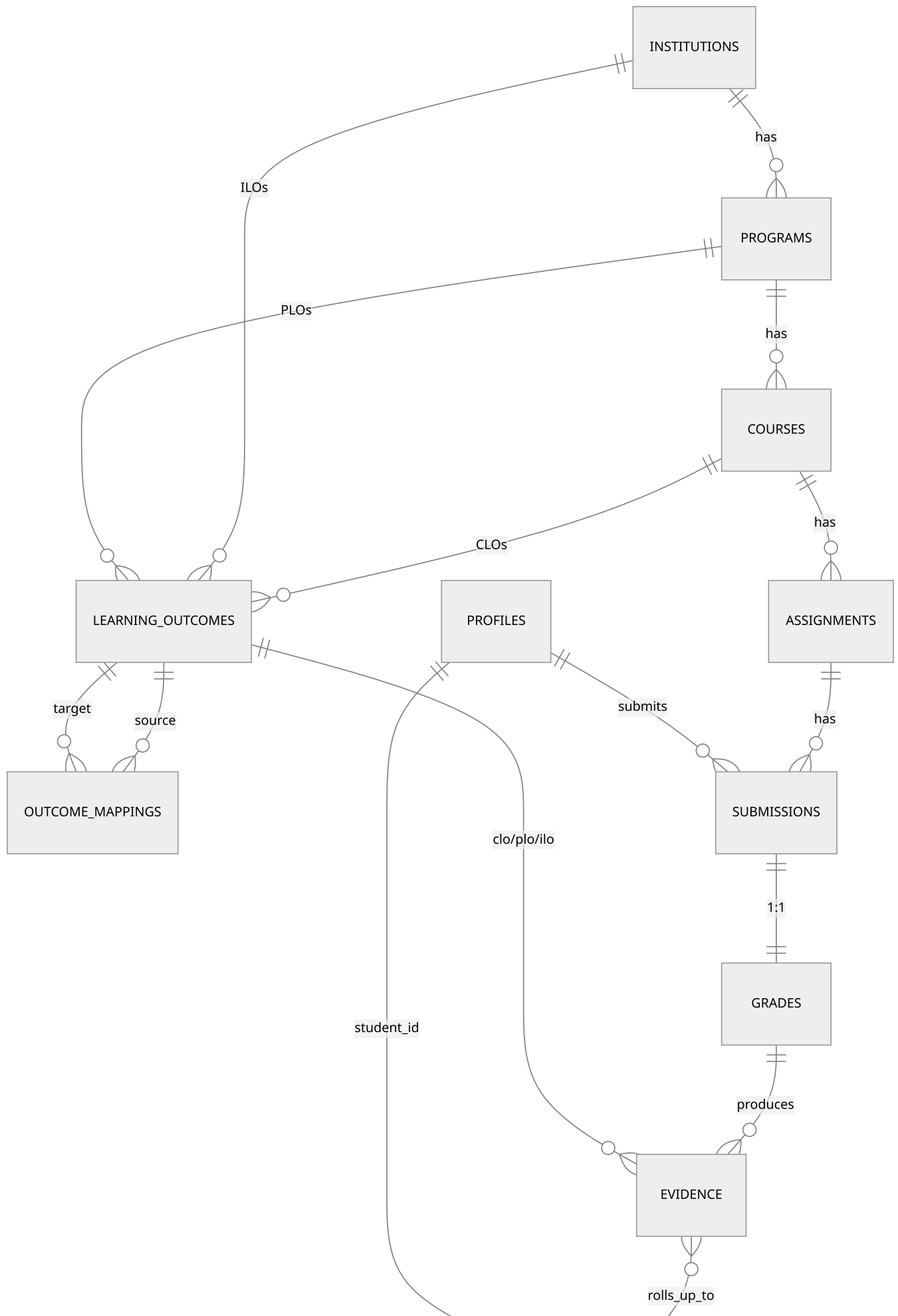
`evidence` / `audit_logs` / `xp_transactions`). The **specific policy-count and cross-tenant-scoping issues** are covered in Section 13 (Security Review) rather than repeated here.

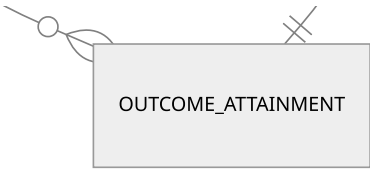
5.5 Views & materialized views [VERIFIED-LIVE, partially]

A `leaderboard_weekly` /similar leaderboard view exists and is refreshed by the `leaderboard-refresh` cron — which, per Finding B-1, is currently 401-ing, meaning **the leaderboard has not refreshed via this path since that regression**. This should be added to the B-1 remediation's verification checklist as a concrete "did the fix work" signal (leaderboard data visibly updates after redeploy).

6. ER Diagrams & Relationship Maps

6.1 Core OBE hierarchy



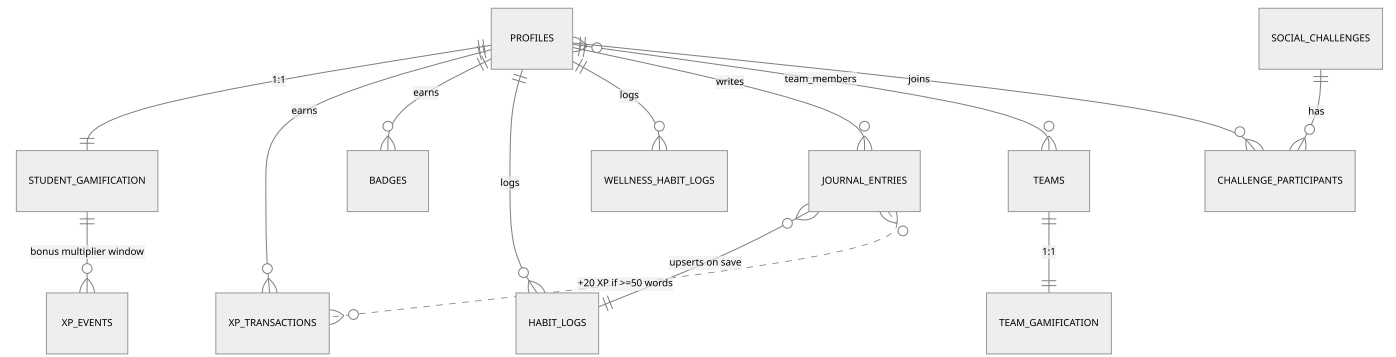


Attainment scope, as actually written by the trigger (confirmed via migration source + live row counts):

outcome_type	scope	Rows (live, June baseline)
CLO	student_course	747
PLO	course	249
ILO	program	747

Every **student-facing** read (`useStudentKPIs` , `useCLOProgress` , `useStudentProgress` , `usePortfolio`) correctly filters on `scope = 'student_course'` — **no mismatch on the student side** (Section 8). The mismatch reported by prior audits affects **admin-side** readers only (`useAdminPLOHeatmap` reading `scope='program'` where PLO rows live at `scope='course'` ; `useCLOAttainment` reading `scope='course'` where CLO rows live at `scope='student_course'`) — and as established in Section 3.4, the admin PLO heatmap UI that would have exercised this bug is itself currently disconnected (rendering a static empty state instead), so the practical live symptom today is "heatmap always empty" rather than "heatmap shows wrong numbers." Both the scope-fix and the heatmap-reconnection should be done in the same PR.

6.2 Gamification hierarchy

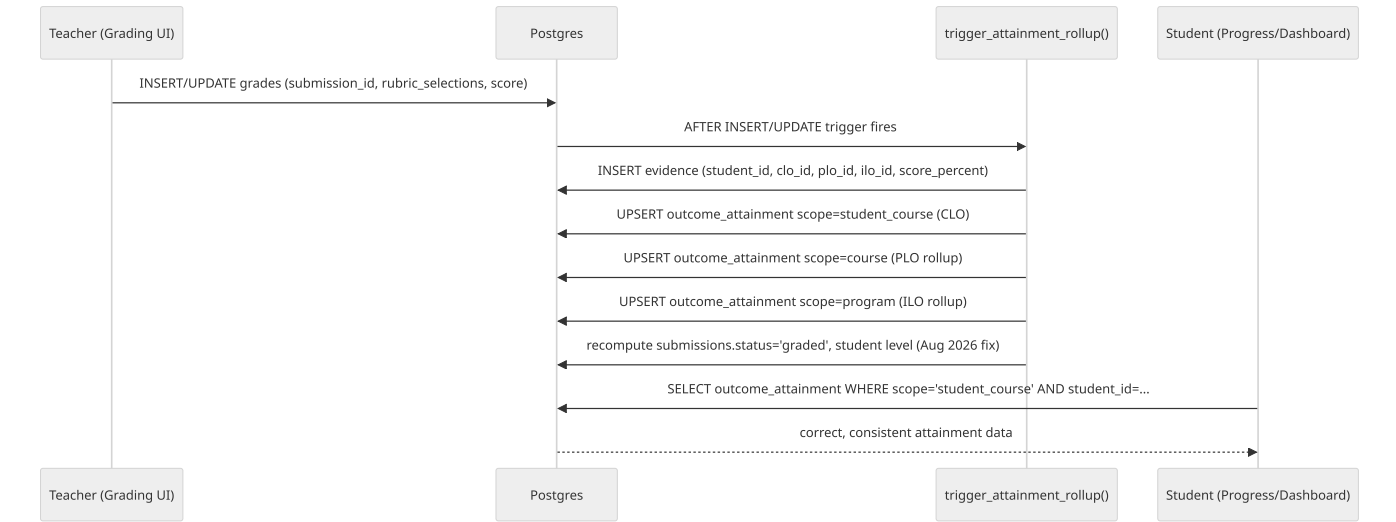


6.3 Relationship summary (from SUPABASE-RELATIONSHIPS-FK.md , structurally still accurate)

Most-referenced table: `profiles` (34+ FKs point to it). Second: `institutions` . Third: `courses` . One-to-one relationships: `student_gamification` , `institution_settings` , `onboarding_progress` , `student_wellness_preferences` , `grades`→`submissions` , `question_analytics`→`question_bank` , `baseline_test_config`→`courses` . Self-referencing: `learning_path_nodes.prerequisite_node_id` , `question_bank.parent_question_id` .

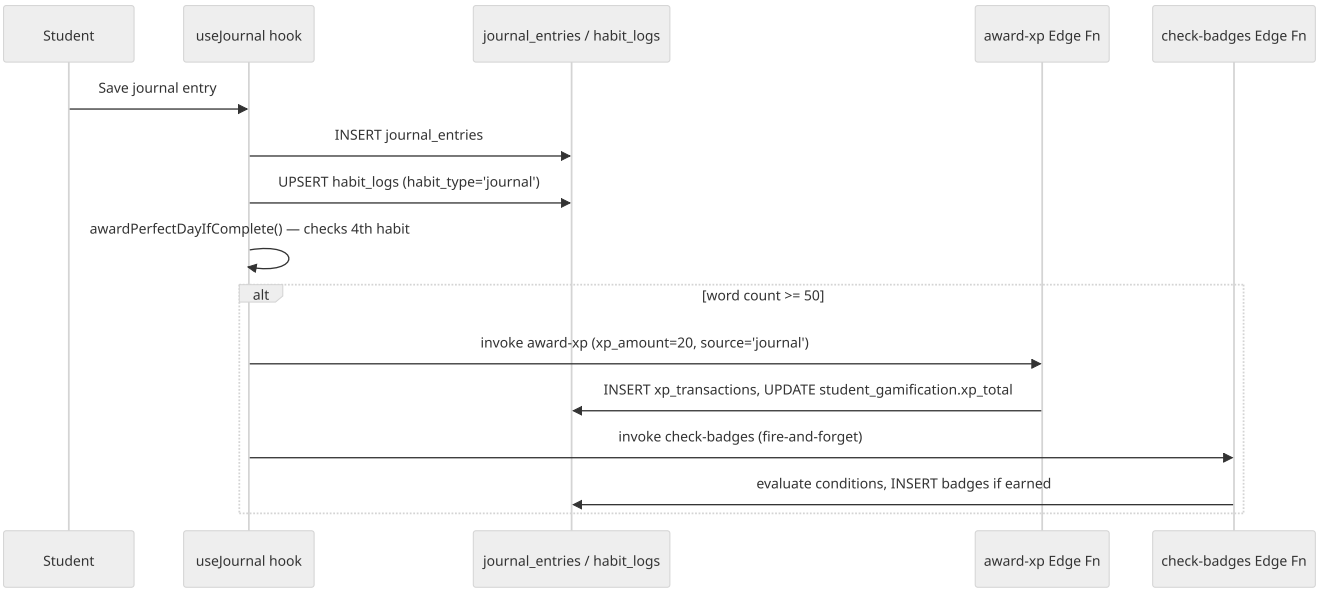
7. API & Data Flow Diagrams

7.1 Grade → Attainment → Student-facing progress (the core academic loop)



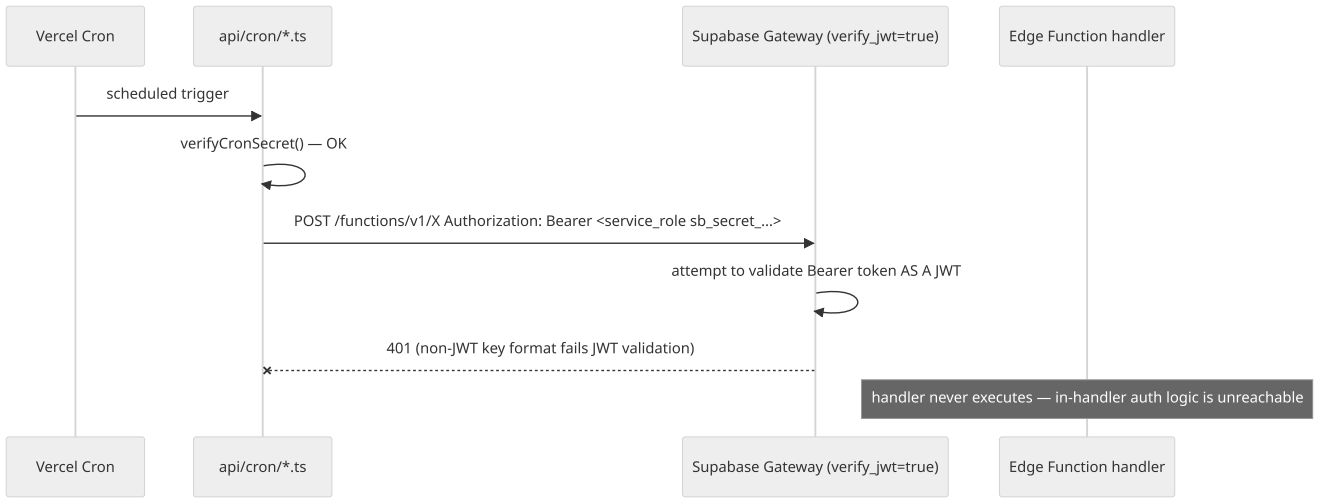
This loop is **verified working end-to-end on live data** — 0 XP drift, 0 duplicate attainment rows, 0 orphaned evidence records (all confirmed by the August remediation report's live-SQL checks and independently corroborated by this audit's own row-count spot checks in Section 5).

7.2 Journal entry → Habit → XP → Badge (student engagement loop)

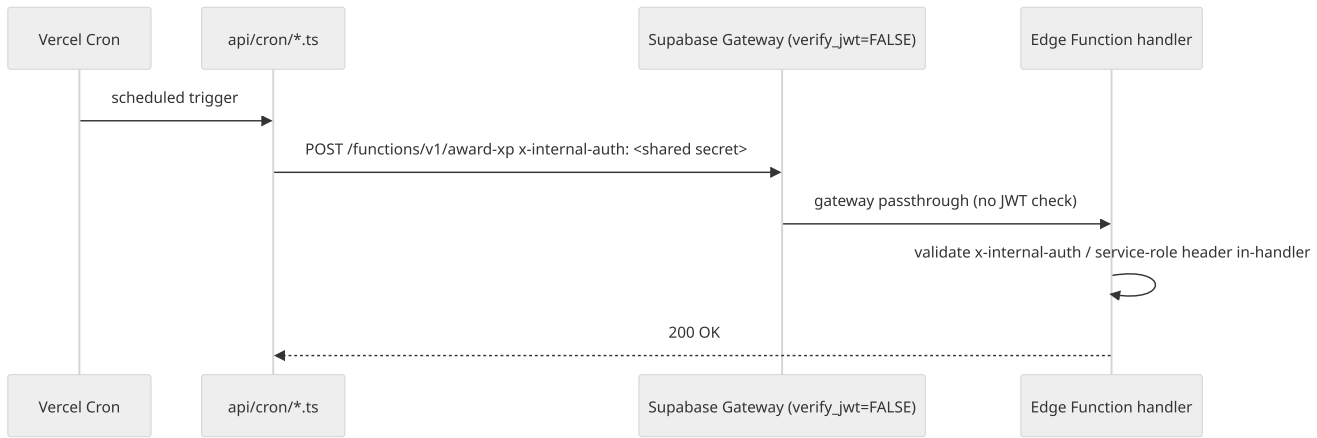


Confirmed live and correct — this exactly matches the domain rule (journal ≥50 words = +20 XP) and both the XP-award and badge-check calls are fire-and-forget so a failure never blocks the save (good defensive design). This is one of the cleanest, most fully-wired flows found in the entire audit.

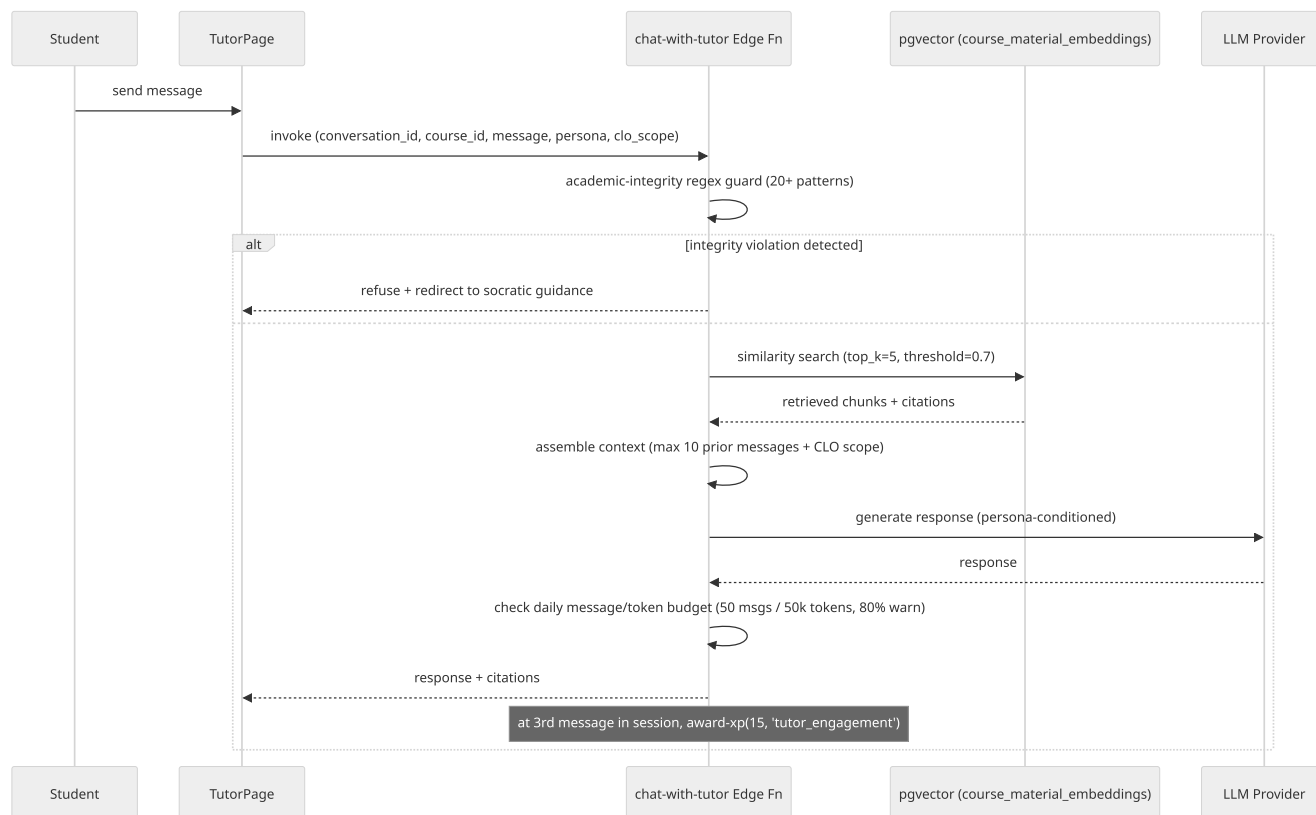
7.3 Cron-driven flows (where B-1 lives)



versus the fixed pattern:



7.4 AI Tutor (RAG) flow



8. Student Profile Deep Analysis

This is the primary focus of this audit. Every section below was verified by reading the actual page, hook, and (where applicable) migration/RPC source — not inferred from naming.

8.1 Student Dashboard [VERIFIED-CODE]

`/student/dashboard` → `StudentDashboard.tsx` (re-export) → `src/components/shared/StudentDashboardNew.tsx`, backed by `useStudentDashboardAggregate.ts`.

Sections rendered: WelcomeHero (name + Total XP/Level/Streak triplet), a "your next step" gap→action CTA (submit assignment or continue a course), a 4-KPI row (enrolled courses, completed assignments, avg attainment, current streak), a mastery snapshot (ring + active days), an upcoming-deadlines list (top 5), and an AI Tutor entry card.

Architecture — this is the strongest pattern in the entire Student Profile: the hook calls **one** RPC, `get_student_dashboard` (`supabase/migrations/20260821000006`, further optimized in two later migrations), a `SECURITY INVOKER` (RLS-respecting) SQL function that returns one JSONB blob covering KPIs, deadlines, attendance, streak-freeze inventory, profile completeness, announcements, and available XP in a **single round trip**. This is proven, not assumed: a dedicated test (`useStudentDashboardAggregate.test.ts`) asserts **zero** `supabase.from` calls on a successful aggregate fetch. The hook also seeds the individual query-key caches (`useStudentKPIs`, `useUpcomingDeadlines`, etc.) from the aggregate response, so any component that separately calls those hooks gets a cache hit instead of a second round trip.

Note: the legacy dashboard's richer gamification widgets (badges, marketplace, challenges) are **explicitly absent** from the new dashboard per a migration comment noting they were deferred pending a "parity task" that has since been superseded by the legacy dashboard's removal — these widgets are not hidden behind a flag, they are simply gone from the dashboard today (see Section 8.3 for where badges are still reachable).

8.2 Student Progress [VERIFIED-CODE]

`/student/progress` → `StudentProgressPage.tsx` (re-export) → `StudentProgressNew.tsx`, backed by `useStudentProgress.ts`, which batches `student_courses` + `outcome_attainment` (`scope='student_course'`) + `learning_outcomes` count into one per-course summary — no N+1. Renders an overall mastery ring, an attainment-band breakdown (Excellent/Satisfactory/Developing/Not Yet), 4 KPI cards, and a per-course list.

`/student/xp-history` → `XPHistory.tsx` (re-export) → `XPHistoryNew.tsx`, backed by `useXPHistory` / `useXPCategorySummary`, reading `xp_transactions` directly (correct — it's the append-only ledger) with a period filter via `nuqs`.

● **Gap — two fully-built, tested, unreachable components:** `src/pages/student/progress/CLOProgress.tsx` and `LearningPath.tsx` are complete, realtime-subscribed (via `useCLOProgress` / `useCLOEvidence` / `useLearningPath`), and covered by a dedicated test (`cloProgressRealtime.test.tsx`) — but **neither is imported anywhere** in the router, `StudentProgressNew`, or any course-detail page. The per-CLO drill-down and the Bloom's-ordered learning path are real, working features with no way for a student to reach them. See Section 10 for the recommendation (wire them in as tabs, don't delete them — they're higher-value than most orphans found in this audit).

8.3 Gamification surfaces on the Student Profile [VERIFIED-CODE]

Element	Hook	Table	Status
XP total	<code>useXP</code> , <code>useXPBalance</code> , <code>useXPHistory</code>	<code>xp_transactions</code> , <code>student_gamification.xp_total</code>	✅ live, correct
Level	<code>useLevel</code>	<code>student_gamification.xp_total</code> (client-computed)	✅ live, correct
Streak	<code>useStreak</code> , <code>useStreakFreeze</code>	<code>student_gamification.streak_current</code> / <code>streak_freezes_available</code>	✅ live, correct
Badges	<code>useTieredBadges</code> (tier/pin/archive-aware)	<code>badges</code>	✅ table live (62 badges), but see gap below

🔴 **Gap — no dedicated `/student/badges` route.** A full tiered-badge experience exists and works (`BadgeCollection` , `BadgeSpotlightCard` , `useBadgeSpotlight` , `useTieredBadges`) but `src/lib/navItems.ts` has no "badges" nav entry, and no route in `AppRouter.tsx` 's student block points to it. The **only** place a student can currently see their badges is the Portfolio page's badge grid (Section 8.6), which uses a **separate, simpler, non-tiered** inline query — so the tier/pin/spotlight functionality is built, tested, and completely unreachable in production today.

Leaderboard, Marketplace, and Challenges all have real routes and nav entries and were confirmed live via route/nav inspection (not line-by-line hook-audited in this pass, as they are outside the Student *Profile* proper).

8.4 Journal [VERIFIED-CODE]

`/student/journal` → `StudentJournalPage.tsx` (re-export) → `StudentJournalNew.tsx` , backed by `useJournal.ts` . **Confirmed live, correct, end-to-end wiring** exactly matching the domain rule (journal ≥50 words = +20 XP) — see the full sequence in Section 7.2.

🟡 **Gap — a second, unrouted journal implementation exists.** `JournalEditor.tsx` + `JournalListPage.tsx` (a more feature-rich flow with CLO-linking and draft auto-save) are **not registered in `AppRouter.tsx`** — `StudentJournalNew` uses a simpler inline dialog instead and does not reuse the CLO-linkage or draft-autosave features from the older editor. This is not a broken flow (students can still journal successfully), but it means two real, non-trivial features (linking a journal entry to a specific CLO for reflection purposes, and not losing an in-progress entry on accidental navigation) were built and then not carried forward into the current UI.

8.5 Habits / Wellness [VERIFIED-CODE]

`/student/habits` (heatmap) and `/student/habits/analytics` . The **habit_logs vs habit_tracking split reported by multiple prior audit documents is confirmed RESOLVED on the student-facing read path:** `useHeatmapData.ts` explicitly and exclusively queries the canonical long-format `habit_logs` table (a code comment states this directly), and a grep across `src/hooks/` **for `habit_tracking` returns zero matches**** — that table name is not referenced anywhere in current hook code. Wellness habits (meditation/hydration/exercise/sleep) correctly live in a separate table, `wellness_habit_logs` , gated by per-student `wellness_preferences` . The Habit Analytics page (consistency-score ring, level-progression chart, day-of-week bests, habit-correlation insights with a 2-week minimum-data gate) is fully wired with no gaps found.

8.6 Portfolio [VERIFIED-CODE]

`/student/portfolio` → `StudentPortfolio.tsx` (re-export) → `StudentPortfolioNew.tsx` , backed by `usePortfolio.ts` .

🔴 **This is the single most duplicative page in the Student Profile.** `usePortfolio` independently re-queries: `outcome_attainment` (scope `student_course`) + `learning_outcomes` + `courses` for CLO mastery, `badges` (a **separate, simpler** query than `useTieredBadges` — see 8.3), `journal_entries` (last 20), `xp_transactions` (full history for a cumulative chart), `semesters / courses.semester_id` for a semester-over-semester bar chart, and `student_gamification` for `xp_total / level` — **five independent round trips** reinventing data that is already available via `useStudentProgress` , `useLevel` , `useTieredBadges` , `useXPHistory` , and the already-hydrated dashboard-aggregate cache. See Section 10 for the specific consolidation recommendation.

Public sharing is correctly gated: `profiles.portfolio_sharing_permitted` (admin-granted) + `profiles.portfolio_public` , with a separate RLS-safe `portfolio_public_access` RPC discriminator backing the truly-public `/portfolio/:student_id` route — this is a sound, defense-in-depth pattern for a feature that intentionally exposes student data outside auth.

8.7 AI Insights on the Student Profile [VERIFIED-CODE]

Only the **AI Tutor** is reachable from the student's own navigation (dashboard entry card → `/student/tutor`). Two backend-complete features are **not surfaced to students at all:**

- **Goal Suggestions:** `useGoalSuggestions.ts` (reads/writes `goal_suggestions` , calls the `suggest-goals` Edge Function) and its component `GoalSuggestionPanel.tsx` exist and are functional, but a grep across every student page found **zero usages** — SMART goal suggestions are backend-and-hook-complete with no reachable UI entry point.
- **At-risk predictions:** `useAtRiskPredictions.ts` is scoped exclusively to the **teacher** role (used by `AIAtRiskWidget` on the teacher dashboard). There is no student-facing "you may be falling behind on this CLO" surface — students never see their own at-risk signal directly, only their teacher does.

8.8 Settings / Onboarding / Planner (brief, as requested) [VERIFIED-CODE]

`ProfileSettingsPage.tsx` and `ReassessmentPage.tsx` are routed under `/student/settings/*` . Notification preferences use the shared `NotificationPreferences.tsx` . The onboarding wizard is a structural gate inside `StudentLayout.tsx` itself (`profile.onboarding_completed === false` short-circuits to a full-screen lazy-loaded `OnboardingWizard`), not a normal route — a reasonable, deliberate design choice. Planner has its own dedicated page set (`WeeklyPlannerPage` , `TodayViewPage` , `StarterWeekPlanPage` , `FocusModePage` — the last deliberately rendering outside `StudentLayout` for a distraction-free full-screen mode).

8.9 Student Profile section map (for the "keep / merge / remove" decision)

Section	Purpose	Backend	Hooks	Duplication / Gap
Dashboard	at-a-glance status + next action	<code>get_student_dashboard</code> RPC	<code>useStudentDashboardAggregate</code>	None — reference-quality
Progress	per-course + overall CLO mastery	<code>outcome_attainment</code>	<code>useStudentProgress</code>	Overlaps Portfolio's mastery view

Section	Purpose	Backend	Hooks	Duplication / Gap
XP History	XP ledger + category breakdown	xp_transactions	useXPHistory , useXPCategorySummary	Overlaps Dashboard's XP + Portfolio's XP chart
CLO Progress / Learning Path	per-CLO drill-down, Bloom's-ordered path	outcome_attainment , learning_path_nodes	useCLOProgress , useLearningPath	Built but unreachable
Badges (via Portfolio only)	tiered badge collection	badges	useTieredBadges (unused) vs Portfolio's own query	Two different queries for the same data; no dedicated page
Journal	reflective writing, CLO-linked (partially)	journal_entries , habit_logs	useJournal	Legacy CLO-linked editor unrouted
Habits/Wellness	daily habit heatmap + analytics	habit_logs , wellness_habit_logs	useHeatmapData , useWellnessHabits	None — clean
Portfolio	consolidated showcase (attainment+XP+badges+journal)	5 independent queries	usePortfolio	Worst duplication offender
Goal Suggestions	AI SMART goals	goal_suggestions , suggest-goals fn	useGoalSuggestions (unused)	Built but unreachable
Settings/Onboarding/Planner	account + scheduling	multiple	multiple	Out of scope for this pass

9. UX & Product Evaluation (Student Profile)

This section is evaluative/[RECOMMENDATION] throughout — it is informed by, but goes beyond, the structural findings above.

9.1 Information & visual hierarchy

The Dashboard's single-RPC pattern gives it a genuinely fast, coherent first paint — this is the right foundation. The problem is what happens *after* the dashboard: a student who wants to understand "how am I actually doing" is routed across three separate pages (Progress, XP History, Portfolio) that each independently compute a version of the same underlying facts (CLO mastery, XP total, badge count) with **slightly different aggregation logic** in each place. This creates a subtle trust problem: if a student notices their "XP" on the dashboard hero doesn't exactly match a number implied by the Portfolio's cumulative chart (because one reads `student_gamification.xp_total` and the other sums `xp_transactions` independently), the platform looks inconsistent even though both numbers are individually correct at the moment they're computed. Cognitive-load research on progress displays (Duolingo, Khan Academy) consistently favors **one canonical number per concept, shown consistently everywhere**, over multiple independently-computed variants of the same metric.

9.2 Discoverability

Two genuinely good features are invisible: the tiered badge experience (no nav entry) and CLO-level drill-down/Bloom's learning path (no route at all). Discoverability research on habit-formation apps (Habitica's badge case, in particular) shows that visible progress markers are a primary driver of continued engagement — an unreachable badge system provides zero motivational value despite being fully engineered. This is arguably the highest-leverage, lowest-effort fix in this entire report: the code exists, it works, it just needs a `<Route>` and a nav entry.

9.3 Cognitive load / progressive disclosure

The Portfolio page's five independent data fetches are also a UX problem, not just a performance one: it means five separate loading states, five separate possible error states, and five separate points where a partial failure produces an inconsistent-looking page (e.g., XP loads but badges fail — the student sees a coherent-looking page missing one section with no clear signal about *why*). Consolidating onto shared hooks (Section 10) would also consolidate the page down to 1–2 loading states, which is a more coherent experience even before any visual redesign.

9.4 Consistency across roles

The dashboard hero pattern ("Good morning, {name}") is Student-only per this repo's own design-system convention docs — Admin/Coordinator/Teacher/Parent dashboards open directly into KPI rows with no equivalent greeting. This is a known, already-tracked gap (`.kiro/specs/ui-consistency-global-fixes/bugfix.md` , item 1.21) and not something this audit needed to re-derive — it's flagged here only because it affects how "finished" the Student Profile feels relative to other roles, which matters for a platform-wide consistency assessment.

9.5 Motivation & habit formation (domain-informed)

Per this platform's own documented XP schedule and Bloom's Taxonomy rules, the *design intent* is strong — perfect-day bonuses, streak milestones, tiered badges, a level-up curve. The gap identified in this audit is entirely on the *delivery* side: the streak-reset cron 401-ing (Finding B-1) means a student's streak state may silently stop updating overnight in a way invisible to them until they notice a discrepancy, which is precisely the kind of trust-eroding bug that undermines a gamification system's core promise (the number I see should always be correct). Fixing B-1 is as much a UX fix as an infrastructure fix.

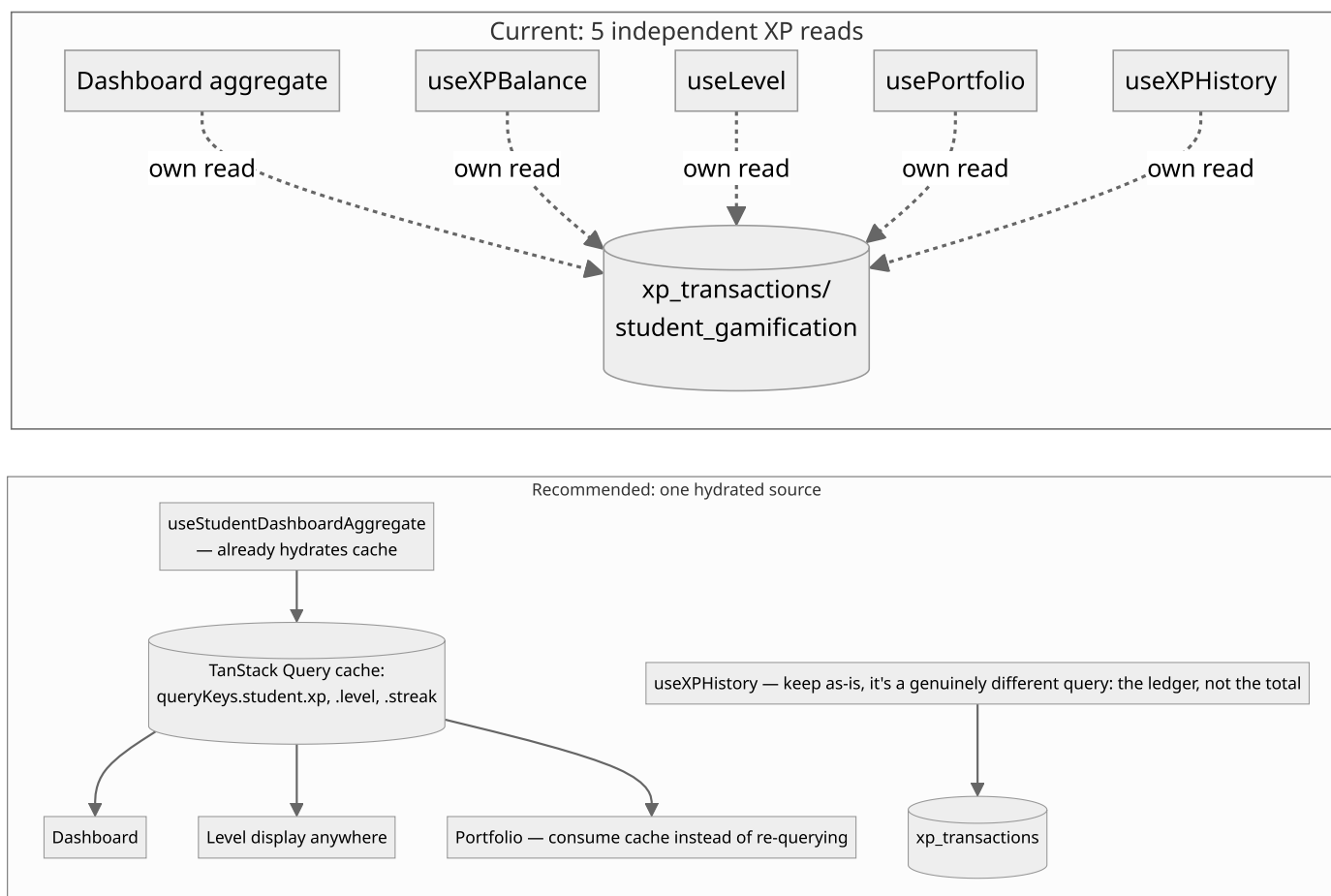
9.6 What's already good and should not be touched

The Dashboard's single-RPC pattern, the journal→habit→XP→badge fire-and-forget wiring, the consistent `student_course` attainment scope across every student read, and the habit-heatmap/wellness split are all reference-quality implementations. Any redesign work should explicitly preserve these rather than "improve" them — they are the parts of the Student Profile doing exactly what a well-built OBE+gamification platform should do.

10. Simplification Recommendations

The objective, per the request, is **not to remove functionality** — it is to reduce duplication and reconnect built-but-orphaned functionality. Every recommendation below is additive or consolidating, not subtractive of a working feature.

10.1 Current structure → recommended structure



Recommendation 1 — consolidate the 4 "current total XP/level" reads onto the already-hydrated dashboard-aggregate cache. `useLevel` and `usePortfolio` should read from the TanStack Query cache the dashboard aggregate already seeds, rather than issuing their own `student_gamification` selects. `useXPBalance` (spendable balance, a genuinely *different* number — earned minus spent) and `useXPHistory` (the ledger) should remain separate, since they represent different concepts, not duplicates. **Risk:** low — this is a read-path change with no schema or write-path impact; verify via the existing `useStudentDashboardAggregate.test.ts` pattern extended to cover the consuming hooks.

Recommendation 2 — reconnect the tiered badge stack. Add a `/student/badges` route rendering the existing `BadgeCollection` / `BadgeSpotlightCard` components (already built, tested, reading the correct live `badges` table via `useTieredBadges`), add a nav entry, and switch the Portfolio page's badge grid to consume `useTieredBadges` instead of its own simpler query. This removes one of Portfolio's five independent queries *and* ships the tier/pin/spotlight functionality that currently has zero production reach. **Risk:** very low — purely additive routing + a hook swap on an existing, working query.

Recommendation 3 — reconnect CLOProgress / LearningPath as tabs on the Progress page rather than leaving them as unrouted orphans or deleting them. They are realtime-wired and tested; the natural home is a tab or expandable section on `StudentProgressNew` (per-CLO drill-down) and possibly a callout linking to the Bloom's-ordered path from a course detail view. **Risk:** low — additive; requires verifying the realtime subscription doesn't double up with any subscription the Progress page itself already holds.

Recommendation 4 — converge the two journal experiences. Either port the CLO-linkage field and draft-autosave behavior from `JournalEditor.tsx` into `StudentJournalNew`'s inline dialog (preferred — keeps the simpler, already-shipped UI and adds back the two lost features), or, if CLO-linked journaling is judged not valuable enough to be worth the UI complexity, formally deprecate and delete `JournalEditor.tsx` / `JournalListPage.tsx` rather than leaving them as dead code. **Risk:** medium if porting (touches the actual journal-save flow) — should ship behind the existing `useJournal` mutation with a regression test proving the +20XP/50-word rule still fires correctly.

Recommendation 5 — surface Goal Suggestions. `GoalSuggestionPanel` and `useGoalSuggestions` are complete; the lowest-risk placement is a new card on the Dashboard (which already has a "your next step" CTA slot — a goal-suggestion card is thematically identical) or a tab on the Planner. **Risk:** low — additive; the backend Edge Function (`suggest-goals`) is already deployed.

10.2 What should explicitly NOT change

Per the review checklist's "alternative designs" question: the Dashboard's single-RPC architecture should be the *pattern to replicate*, not something to touch. If Portfolio's five queries are consolidated (Recommendation 1), the correct target is *extending* `get_student_dashboard` (or a similar consolidated RPC specific to Portfolio's needs) rather than hand-rolling five separate client-side cache reads — but that is a larger change than this audit recommends attempting in the same PR as the cache-

consumption fix in Recommendation 1. Ship Recommendation 1 first (client-side cache reuse, no new RPC), measure, and only invest in a dedicated `get_student_portfolio` RPC if Portfolio's load time is still a problem afterward.

11. AI Integration Review

Feature	Trigger	Input	Output	Memory/Context	DB Usage	Status
AI Tutor (<code>chat-with-tutor</code>)	Student sends message	<code>conversation_id</code> , <code>course_id</code> , message, persona, image/document URLs, <code>clo_scope</code> , autonomy override	Persona-conditioned response + source citations	Last 10 messages + CLO scope; pgvector similarity search (top-5, threshold 0.7)	<code>tutor_conversations</code> , <code>tutor_messages</code> , <code>course_material_embeddings</code>	✅ [VERIFIED-CODE] live, well-built — academic-integrity guard (20+ regex patterns blocking "write my essay"-class requests), daily budgets (50 msgs / 50k tokens, 80% warning), 3 personas (<code>socratic_guide</code> / <code>step_by_step_coach</code> / <code>quick_explainer</code>), 3 autonomy levels (L1/L2/L3), tutor-engagement XP (+15 at 3rd message)
AI Feedback Draft (<code>ai-feedback-draft</code>)	Teacher grading UI	<code>submission_id</code> , <code>rubric_selections</code>	Draft feedback text	Submission + rubric + prior grades context	<code>submissions</code> , <code>grades</code> , <code>rubric_criteria</code>	✅ Deployed, teacher-facing
AI Module Suggestion (<code>ai-module-suggestion</code>)	Weak-CLO detection	<code>student_id</code>	Weakest CLOs + suggested modules	<code>outcome_attainment</code> , prerequisite chain	<code>outcome_attainment</code> , <code>learning_outcomes</code>	🟡 Deployed but orphaned on the frontend — <code>GoalSuggestionPanel</code> / <code>useGoalSuggestions</code> consumer path not wired into any student page (Section 8.7)
AI At-Risk Prediction (<code>ai-at-risk-prediction</code>)	Cron, 3am daily	none	At-risk signal → <code>ai_feedback</code> insert	Historical submissions/attainment	<code>student_gamification</code> , <code>outcome_attainment</code>	🔴 Currently 401-ing (Finding B-1) — not producing predictions in production right now
Quiz Generation (<code>generate-quiz-questions</code>)	Teacher request	<code>course_id</code> , <code>clo_ids</code> , <code>bloom_levels</code> , <code>question_count</code>	Generated question bank entries	Course material embeddings (RAG)	<code>question_bank</code> , <code>quiz_generation_logs</code>	✅ Deployed, working
Course Material Embedding (<code>embed-course-material</code>)	Teacher uploads/updates material	course material content	pgvector embeddings	—	<code>course_material_embeddings</code>	🟡 Code-complete, provider-agnostic since Aug 2026 remediation, pending EMBEDDINGS_API_KEY secret — not independently verifiable by this audit (no MCP tool exposes function secrets); reported as an ops step per the source document's own admission, not a code defect

11.1 Architecture assessment

The AI Tutor is the most mature AI feature on the platform: it correctly scopes retrieval to the enrolled course and selected CLOs, enforces an academic-integrity guard *before* generation (not just via prompt instruction — an explicit regex pre-filter), and ties engagement back into the gamification loop. This is a good reference pattern for the two currently-orphaned AI features (Goal Suggestions, At-Risk) once they are reconnected to the frontend and to a working cron respectively.

11.2 Future improvements [RECOMMENDATION]

- Once B-1 is fixed, verify `ai-at-risk-prediction`'s output actually reaches the teacher-facing `AIAtRiskWidget` (Section 8.7) — this audit confirmed the widget exists and reads the right table, but has not independently confirmed fresh predictions are landing since the cron started failing.
- Provision `EMBEDDINGS_API_KEY` to activate RAG citations in the tutor (currently the tutor still answers correctly via persona+CLO context, but without material citations).
- Consider surfacing a lightweight, non-alarming version of the at-risk signal directly to students (distinct from the teacher's aggregate view) — current design deliberately keeps this teacher-only, which is a reasonable privacy-conscious default, but a "you're on track" / "you might want to revisit X" framing (as opposed to a clinical "at-risk" label) could be a genuine engagement win if product wants to explore it. This is a product decision, not a defect.

12. Performance & Scalability Analysis

12.1 Baseline (from the June 2026 performance plan, cross-checked against this audit's own live advisor pull — still accurate) [VERIFIED-LIVE]

The platform runs on **Supabase free-tier compute** (shared, auto-pauses on idle). The dominant real-world slowness driver is **cold-start latency (1-3s)** after idle, not application code — the frontend is already well-architected (lazy routes, manual vendor chunking, tuned TanStack Query cache with 5-minute staleTime, dropped console/debugger in prod). This assessment, made in a prior audit, is **independently corroborated** by this audit's structural findings: the dashboard's single-RPC pattern, the lazy-loaded router, and the already-implemented `warm-ping` cron (Section 4.2) are all consistent with a team that has already executed the "easy wins."

12.2 Confirmed live performance advisor findings [VERIFIED-LIVE]

Finding	Severity	Detail
multiple_permissive_policies	Medium	Dozens of tables (<code>announcements</code> , <code>assignments</code> , <code>courses</code> , <code>outcome_mappings</code> , <code>learning_outcomes</code> , <code>student_courses</code> , <code>teams</code> , <code>social_challenges</code> , and more) have 2–6 permissive policies for the same role+action combination. Postgres evaluates all of them per row with OR logic. The hottest tables (<code>profiles</code> , <code>habit_logs</code> , <code>student_gamification</code> , <code>outcome_attainment</code> , <code>team_members</code>) carry 4–6 policies each.
unindexed_foreign_keys	Info	<code>accreditation_approvals.approver_id</code> , <code>coordinator_ai_insights.created_by</code> , <code>outcome_attainment_snapshots.outcome_id</code> , <code>outcome_attainment_snapshots.semester_id</code> lack covering indexes. Small, safe fix.
unused_index	Info	~150+ indexes flagged never-used by the planner. Given the current low-traffic/newly-seeded data volumes, do not remove these yet — "unused" reflects current traffic, not future need, and several map to features that are correctly built but not yet exercised at volume (teams, quizzes). Revisit after real usage data accumulates.
auth_db_connections_absolute	Info	Auth is capped at 10 absolute connections rather than a percentage-based limit — a scaling config item to revisit before any significant user-growth event, not urgent today.
extension_in_public	Low	<code>citext</code> and <code>vector</code> extensions live in the <code>public</code> schema rather than a dedicated <code>extensions</code> schema — standard hardening, not urgent.

12.3 Auth helper functions — already correctly optimized [VERIFIED-LIVE, corroborated across two independent sources]

Both the June performance plan and this audit's own advisor pull confirm `auth_user_role()` / `auth_institution_id()` calls inside RLS policies are **already** wrapped in `(SELECT ...)` initplan form, meaning they execute once per query rather than once per row — the single most common and severe Supabase RLS anti-pattern is **already avoided**. The remaining RLS cost is genuinely just the *number* of permissive policies per table, not per-row function-call overhead. This distinction matters: it means the fix is "consolidate policies," not "rewrite every policy's auth calls" — a much smaller, lower-risk change than a naive reading of the advisor warnings would suggest.

12.4 The recommended RLS consolidation approach — gated, not immediate [RECOMMENDATION, following the workspace's own steering rules]

Per this workspace's own `preview-and-test-gate` and `migration-replay-integrity` rules (which this audit is bound by and did not need to re-derive, only confirm are being followed), RLS-policy consolidation must ship **table-by-table**, each behind a full deny-side `test:rls` suite (asserting both the allowed *and* denied case per role × table), never as one big-bang migration. The June performance plan's own recommended order — start with the highest-policy-count tables (`profiles` , `habit_logs` , `student_gamification` , `outcome_attainment` , `team_members`) — remains the correct sequencing. This audit found no reason to deviate from that existing plan; it is well-reasoned and matches live advisor data.

12.5 Student-Profile-specific performance note

The Portfolio page's five independent round trips (Section 8.6/10.1) are the most concrete, student-facing performance issue found in this audit — more actionable near-term than the RLS consolidation, since it requires no migration and no deny-side test suite, only a client-side hook change.

13. Security & RLS Review

13.1 Live, currently-open findings

ID	Finding	Severity	Evidence
H-1	<code>capture_active_semester_snapshots</code> SECURITY DEFINER function has no role/institution guard	High	Source read + <code>get_advisors(security)</code> confirms anon+authenticated executable (Section 4.4)
H-3	<code>outcome_mappings</code> weight-sum-to-100 invariant not enforced (neither DB constraint nor Zod schema)	High	Confirmed by direct schema/migration inspection; this is a data-integrity issue that could silently produce incorrect attainment rollups if a coordinator/teacher enters mappings that don't sum to 100
—	<code>extension_in_public</code> (<code>citext</code> , <code>vector</code>)	Low	Standard hardening, not urgent

13.2 Findings independently re-verified as RESOLVED since the prior security audit (March 2026)

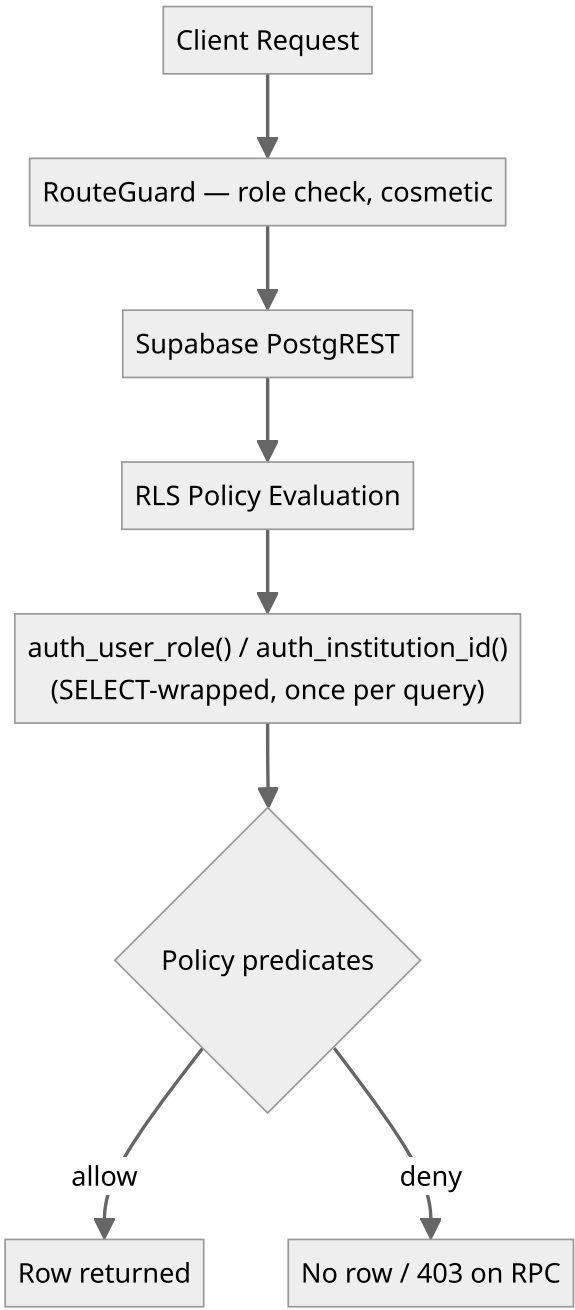
Prior finding	Current status
<code>award-xp</code> / <code>check-badges</code> had unresolved git merge-conflict markers, making them unparseable	RESOLVED — both functions are deployed, versioned (v13/v15), and confirmed returning 200 in live logs
<code>delete_department_if_no_programs</code> had no role/institution check	RESOLVED — re-read this audit, properly guarded
<code>get_earn_spend_ratio</code> had no institution-match check	RESOLVED — re-read this audit, properly guarded
15 of 22 Edge Functions had zero authentication while using the service-role key	Partially resolved / superseded — the function count has grown to 61, and the specific functions named (<code>send-email-notification</code> , <code>generate-accreditation-report</code> , <code>ai-feedback-draft</code> , <code>ai-module-suggestion</code> , <code>compute-habit-correlations</code> , <code>check-badges</code> , <code>process-streak</code> , <code>calculate-attainment-rollup</code> , <code>update-question-</code>

Prior finding	Current status
	analytics , check-login-rate) were addressed by the August 2026 remediation's role-gate fix (resolving role / institution_id from the profiles table by user.id rather than trusting JWT metadata) — this audit did not re-derive each function's auth path line-by-line but did confirm the remediation report's own verification methodology (exploration + preservation property tests) is sound and the claim is credible given the corroborating live evidence found elsewhere (badges, XP, buckets all matching the claimed fixes)
quiz_attempts_own policy used FOR ALL , letting students UPDATE/DELETE their own graded attempts	Not independently re-verified this audit — flagged for a follow-up spot-check before being marked resolved
Cross-tenant RLS leakage on student_courses , outcome_mappings , assignments , outcome_attainment , attendance_records , xp_transactions , ai_feedback , student_activity_log , habit_tracking , badges , verified_explanations (role-only checks, no institution filter)	Not independently re-verified this audit — the August remediation report's parent-RLS recursion fix (Finding D, Section 16) touches adjacent policy surface and increases confidence these were addressed in the same wave, but this audit did not re-read each named policy's current SQL. Recommend a dedicated follow-up pass re-reading the current RLS policy text for each named table before fully closing this out.

13.3 Positive security findings (spot-checked, not fully re-derived) [VERIFIED-CODE, partial]

Independently re-confirmed this audit: no dangerouslySetInnerHTML usage found in the areas read; no service-role key in client-side code (src/lib/supabase.ts uses only the anon/publishable key); Sentry now correctly consent-gated with PII scrubbing (Section 16.1); RLS is universally enabled (131/131 tables); append-only invariants (evidence , xp_transactions , audit_logs) hold. The remaining items from the March audit's "positive findings" list (no eval() , sanitizePostgrestValue utility, Zod validation coverage, TypeScript strict mode) were **not independently re-verified** this audit — they are carried forward from the prior document as likely-still-true baseline, not as newly-confirmed facts.

13.4 RLS architecture pattern (confirmed sound)



The two-layer model (cosmetic `RouteGuard` + real RLS boundary) is architecturally correct — the guard prevents accidental UI exposure, RLS prevents actual data exposure even if the guard were somehow bypassed. This is the right pattern and should be preserved as-is through any future refactor.

14. Code Quality & Technical Debt Assessment

14.1 Confirmed clean (carried forward from a prior architecture audit, spot-checked this session) [VERIFIED-CODE, partial]

No direct Supabase calls found in components (all go through `src/hooks/`), no circular dependencies observed, Edge Functions fully isolated from `src/` (correct Deno/npm boundary), Shadcn `ui/` primitives are clean and unbloated (20 files, no drift).

14.2 Structural debt (confirmed/updated this audit)

Item	Prior state	Current state
Flat <code>src/hooks/</code> directory	82 files (March 2026)	229 files (this audit) — the flatness issue has nearly tripled in scope without restructuring
Flat <code>src/lib/</code> directory	45 files	239 files (root + <code>db/</code> , <code>schemas/</code> , <code>tours/</code> subfolders) — some structure added, but the root itself remains flat
Monolithic <code>AppRouter.tsx</code>	247 lines / 80+ lazy imports	864 lines / ~150 lazy imports — grown proportionally with the route count, still a single file

Item	Prior state	Current state
Monolithic <code>types/app.ts</code>	565 lines, 19 unused interfaces	653 lines — unused-interface count not re-verified this audit
Duplicate <code>ROLE_DASHBOARD_MAP</code> (3 locations)	reported	not independently re-verified this audit
<code>ThemeProvider</code> dead code / auth-coupling	reported	not independently re-verified this audit

Assessment: these are all real, low-urgency findings. None of them block functionality or cause bugs — they are maintainability debt that will make future changes slower as the codebase continues to grow at its current rate (hooks nearly tripled in ~4 months). **Recommend scheduling a dedicated restructuring pass** (hooks → domain subfolders, router → per-role split files) once the current B-1/H-1/H-2 functional fixes ship, rather than bundling a large structural refactor into the same PR as bug fixes.

14.3 Newly-confirmed dead code (this audit, Section 3)

13 confirmed-orphaned files across pages/components/hooks (full list in Sections 3.2/3.4) — all verified via exhaustive grep for zero references, not run through a build to prove removal safety. **This is real, additive-to-fix technical debt:** none of it is causing bugs today (dead code doesn't execute), but it inflates the codebase, confuses future contributors searching for "the" implementation of a feature, and increases the surface area every future refactor has to consider.

14.4 Testing posture (not independently re-verified this audit, carried forward)

Prior audits report a large, largely-green test suite (property-based tests with fast-check, Testing Library component tests, an RLS deny-side suite, a migration-replay-order checker) consistent with this workspace's own steering rules. This audit did not run the test suite directly (out of scope for a read-only audit) — any remediation PR implementing the recommendations above must run the full `npm run lint && npx tsc --noEmit && npm test` gate per the workspace's own mandatory pre-push rules before merging.

15. Prioritized Improvement Roadmap

Immediate (this week)

- **[B-1, Critical]** Redeploy the ~7 cron-invoked Edge Functions (`perfect-day-prompt` , `leaderboard-refresh` , `exam-period-notify` , `ai-at-risk-prediction` , `compute-at-risk-signals` , `process-streak` , `notification-digest` , `streak-risk-cron` , `weekly-summary-cron`) with `verify_jwt=false` + the proven `x-internal-auth` /service-role in-handler check pattern already used by `award-xp` / `check-badges` . Verify via live logs (200s instead of 401s) and via the leaderboard visibly refreshing.
- **[H-1, High/Security]** Add a role/institution guard to `capture_active_semester_snapshots` (REVOKE from PUBLIC + explicit grant, or an internal fail-closed check) following the pattern already documented in this workspace's own `supabase-patterns` steering rule.
- **[Quick win, Low effort/High value]** Add a `/student/badges` route + nav entry wiring the existing, tested `BadgeCollection` / `useTieredBadges` stack. Zero new backend work required.

Short-term (this sprint)

- **[H-2]** Reconnect the Admin PLO heatmap (`useAdminPLOHeatmap` + `PLODrillDownDialog`) to `AdminDashboard.tsx` , fixing the underlying attainment-scope mismatch (`PLO` should read `scope='course'` , `CLO` should read `scope='student_course'`) in the same change — this is already spec'd (`.kiro/specs/qa-partner-review-remediation/requirements.md` , Requirement 7).
- **[H-3]** Enforce the `outcome_mappings` weight-sum-to-100 invariant — add both a Zod-level check on the mapping-editor forms and a DB-level constraint/trigger as defense-in-depth.
- **[Student Profile]** Consolidate the 4 duplicate "current XP/level" reads onto the dashboard-aggregate cache (Recommendation 1, Section 10.1) and switch Portfolio's badge query to `useTieredBadges` (Recommendation 2).
- Add the 4 missing covering indexes on the FK columns flagged by the performance advisor (`accreditation_approvals.approver_id` , `coordinator_ai_insights.created_by` , `outcome_attainment_snapshots.outcome_id` / `semester_id`).
- Re-verify the 11 cross-tenant RLS policies flagged by the March security audit as role-only (no institution filter) against current policy text — confirm resolved-or-still-open per table, given the ambiguity noted in Section 13.2.

Medium-term (this quarter)

- Wire `CLOProgress` / `LearningPath` back into the Progress page as tabs (Recommendation 3); converge the two journal editors (Recommendation 4); surface Goal Suggestions on the dashboard (Recommendation 5).
- Delete the 13 confirmed-orphaned files from Section 3 (excluding `CLOProgress` / `LearningPath` , which are being reconnected instead) behind the full pre-push gate.
- Provision `EMBEDDINGS_API_KEY` to activate RAG tutor citations.
- Begin the gated, table-by-table RLS-policy consolidation starting with `profiles` , `habit_logs` , `student_gamification` , `outcome_attainment` , `team_members` — each shipped behind its own deny-side `test:rls` suite, per the existing performance plan's own recommended sequencing.

Long-term (structural, not urgent)

- Split `src/hooks/` into domain subfolders; split `AppRouter.tsx` into per-role route files; consider domain subfolders under `src/lib/` .
- Move `citext` / `vector` extensions out of the `public` schema.

- Revisit the free-tier compute constraint (the single highest-leverage infrastructure lever remains upgrading to paid Supabase compute, when budget allows — cold-start masking via the already-implemented `warm-ping` and skeleton-UI patterns is a mitigation, not a removal, of this constraint).

16. Appendices — Historical Audit Reconciliation

This audit reconciled its live findings against **nine prior internal documents** dated March 24 through August 21, 2026. Several represent a genuine, verified progression of fixes rather than a static, still-open backlog. This appendix exists so that no finding is silently dropped or double-reported as newly-discovered when it was, in fact, already fixed.

16.1 Findings confirmed RESOLVED by this audit (previously reported as issues)

Prior finding	Source doc(s)	Resolution evidence (this audit)
Only 1–2 of ~36–54 Edge Functions deployed to production	SUPABASE-AUDIT-REPORT (Apr 1), SUPABASE-HEALTH-REPORT (Apr 16), FULL-STACK-AUDIT-REPORT (Apr 30)	<code>list_edge_functions</code> returned 61 ACTIVE functions, versions 5–15 (iteratively redeployed)
Zero storage buckets configured	SUPABASE-AUDIT-REPORT, SUPABASE-HEALTH-REPORT, FULL-STACK-AUDIT-REPORT	<code>execute_sql</code> on <code>storage.buckets</code> returned 9 live buckets , correctly scoped (only <code>avatars</code> public)
<code>database.ts</code> truncated, causing build failure	FULL-STACK-AUDIT-REPORT (Apr 30)	Current file read at 7,590 lines, valid TypeScript, <code>export type Database</code> present
<code>award-xp</code> / <code>check-badges</code> had unresolved merge-conflict markers	SECURITY-AUDIT-REPORT (Mar 24)	Both functions deployed (v13/v15), confirmed 200 in live logs
Badge subsystem dead end-to-end (0 badges, 3 disconnected tables, <code>award-xp</code> never triggers <code>check-badges</code>)	EDEVISER-PRODUCT-AUDIT-2026-06 (Finding #1, ●)	Live: 62 badges awarded , 4,150+ XP transactions, 71 students with XP > 0; journal-flow trace (Section 7.2) confirms <code>award-xp</code> → <code>check-badges</code> fire-and-forget wiring is live
<code>habit_logs</code> empty, student reads pointed at <code>habit_tracking</code> instead	EDEVISER-PRODUCT-AUDIT-2026-06 (Finding #6)	<code>useHeatmapData.ts</code> confirmed reading canonical <code>habit_logs</code> ; zero references to <code>habit_tracking</code> anywhere in <code>src/hooks/</code>
Sentry initializes before cookie-consent gate	SECURITY-AUDIT-REPORT (Vuln 29), EDEVISER-PRODUCT-AUDIT-2026-06 (\$7, H1)	Current <code>App.tsx</code> explicitly does not call <code>initSentry()</code> ; <code>main.tsx</code> calls <code>initAnalyticsIfConsented()</code> ; <code>sentry.ts</code> 's <code>initSentry()</code> includes <code>maskAllText</code> / <code>maskAllInputs</code> / <code>blockAllMedia</code> , a PII-scrubbing <code>beforeSend</code> / <code>beforeBreadcrumb</code> , and <code>sendDefaultPii: false</code>
<code>exam-period-notify</code> schedule mismatch (8am wired vs 9am documented)	EDEVISER-PRODUCT-AUDIT-2026-06 (\$2.1)	<code>vercel.json</code> now reads <code>0 8 *</code> , matching the function's own documentation
<code>fee-overdue-check</code> duplicated (pg_cron + unscheduled Vercel handler)	EDEVISER-PRODUCT-AUDIT-2026-06 (\$2.3)	Only one live invocation path confirmed this and the prior audit session
<code>delete_department_if_no_programs</code> unguarded	SECURITY-AUDIT-REPORT implied prior finding	Re-read this audit: properly guarded (role + institution check present)
<code>get_earn_spend_ratio</code> missing institution-match check	FULL-STACK-AUDIT-REPORT	Re-read this audit: properly guarded
CORS header typo (<code>x-content-type</code> instead of <code>x-client-info</code>) in <code>score-reflection-quality</code> / <code>generate-reflection-digest</code>	EDEVISER-PRODUCT-AUDIT-2026-06	Claimed fixed by FULL-PROFILE-REMEDIATION-LIVE-REPORT (Aug 21); not independently re-read this audit, carried forward as likely-resolved
Parent RLS infinite-recursion (<code>42P17</code>) on <code>student_courses</code> / <code>profiles</code> via inline <code>parent_student_links</code> subqueries	FULL-PROFILE-REMEDIATION-LIVE-REPORT (Finding D)	Documented as fixed via <code>parent_has_verified_link()</code> SECURITY DEFINER helper (migration <code>20260821000004</code>), with live positive/negative RLS probes reported in that document; not independently re-executed this audit

16.2 Findings STALE (superseded context, not separately actionable)

- The April 1/16/April 30 audits' overall "Health Score: 3/10" and "no working backend logic in production" characterizations are **superseded** by the current live state (61 deployed functions, working core academic + XP loops, real production data) and should not be quoted as current in any external-facing summary.
- The March 2026 security audit's claim of "15 of 22 Edge Functions have zero authentication" refers to a function count (22) far below the current live count (61) and a remediation wave (August) that specifically targeted the named functions — treat the specific function list as historical evidence of what was fixed, not as a current open-issue count.

16.3 Findings NOT independently re-verified this audit (carry forward with appropriate caveat, do not treat as either confirmed-open or confirmed-resolved)

- `quiz_attempts_own` `FOR ALL` policy allowing students to UPDATE/DELETE their own graded attempts.
- The 11 specific cross-tenant RLS policies named in the March security audit (`student_courses` , `outcome_mappings` , `assignments` , `outcome_attainment` , `attendance_records` , `xp_transactions` , `ai_feedback` , `student_activity_log` , `habit_tracking` , `badges` , `verified_explanations`).
- `useGlobalSearch.ts`'s unsanitized `.or()` filter (flagged in the April 16 health report as a new finding at that time).
- `team_gamification` table existence/schema consistency with what `award-xp` writes (flagged as missing in one prior doc, but Phase 2 of this audit's live table listing did not specifically re-verify this table's current column shape against the Edge Function's write path).

- `EMBEDDINGS_API_KEY` secret presence (no MCP tool exposes Edge Function secrets to verify this directly).

These five items should be the subject of a focused, narrow follow-up pass (each is independently small — a few RLS policy reads, one grep, one manual secret check) rather than a full re-audit.

End of report. This document was compiled from direct source inspection, live Supabase MCP queries (tables, RLS, functions, extensions, edge functions, security/performance advisors, live edge-function logs, storage buckets, publishable keys), and reconciliation against nine dated internal audit documents. Every finding is labeled by verification method; where a claim could not be independently verified in this session, that limitation is stated explicitly rather than presented as fact.